LAPORAN PRAKTIKUM
INHERITANCE, GENERALIZATION & OVERRIDING

Dibuat untuk memenuhi salah satu tugas Praktikum Mata Kuliah Pengembangan Berorientasi Objek



Dibuat Oleh:

NAMA	: Rena Fiantina
NIM	: 251511028
KELAS	: 2-A
PROGRAM STUDI	: D3 Teknik Informatika

TEKNIK KOMPUTER DAN INFORMATIKA
POLITEKNIK NEGERI BANDUNG
202

A. Praktikum I – Circle & Cylinder

a. Pembuatan Class Circle

Code :

```
public class Circle {
    private double radius;
    private String color;

    public Circle() {
        radius = 1.0;
        color = "red";
    }

    public Circle(double radius) {
        this.radius = radius;
        color = "red";
    }

    public double getRadius() {
        return radius;
    }

    public double getArea() {
        return radius * radius * Math.PI;
    }

    @Override
    public String toString() {
        return "Circle[radius=" + radius
                + ", color=" + color + "]";
    }
}
```

Output & Hasil Percobaan :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac Circle.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac TestCircle.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCircle
c1 radius = 1.0
c1 area = 3.141592653589793
Circle[radius=1.0, color=red]

c2 radius = 2.5
c2 area = 19.634954084936208
Circle[radius=2.5, color=red]
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Program menguji dua objek, yaitu c1 yang dibuat menggunakan constructor yanpa parameter dan c2 yang dibuat menggunakan constructor dengan parameter 2,5.
Dari output yang muncul c1 memiliki radius 1,0 karena dibuat menggunakan constructor tanpa parameter nilai awal. Sedangkan objek c2 memiliki radius 2,5 karena nilai tersebut dimasukan ketika objek dibuat. Sehingga kedua onjek juga menghasil nilai luas lingkaran yang berbeda.

Checkpoint 1: *Tuliskan: mana state Circle? mana behavior Circle? apa perbedaan state dari dua object yang dibuat?*

⇨ Radius dan color merupakan state Circle karena keduanya menyimpan kondisi objek. Kemudian getRadius(), getArea() dan toString(), merupakan method yang mendefinisikan behavior objek.
State c1 memiliki radius 1.0 dan sudah menjadi nilai default, sedangkan c2 memiliki nilai radius yang dapat ditentukan Ketika objek dibuat.

b. Encapsulation pada Circle

Code :

```
    public Circle(double radius, String color) {
        this.radius = radius;
        this.color = color;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }
```

TestCircle.java :

```
        System.out.println();
        c3.setRadius(5.0);
        c3.setColor("purple");
```

Output & Hasil Pengujian :



Membuat objek circle c3 dengan radius 3,0 dan warna biru. Kemudian nilai radius dan warnanya diubah menggunakan method setter.

Checkpoint 2 : *Mengapa code di luar Circle tidak mengubah radius secara langsung? Apa peran method public terhadap state private**?***

⇨ Karena radius merupaka field private, sehingga code diluar class Circle tidak dapat mengubah dan mengaksesnya secara langsung. Maka method getRadius dan setColor akan menjadi perantara untuk mengakses dan mengubah state private tersebut

c. Pembuatan & Pengujian Inheritance Cylinder

Prediksi Singkat :

- Apakah radius dan color perlu ditulis ulang sebagai field Cylinder?
  - ⇨ Tidak perlu, karena atribut radius dan color sudah dimiliki oleh class Circle. Dengan menggunakan extends Circle, class Cylinder dapat menggunakan properti yang sudah ada pada Circle dan bisa diakses oleh subclass.
- Behavior apa dari Circle yang masih berguna?
  - ⇨ Method getRadius() dan getArea() masih digunakan untuk mengetahui jari-jari alas silinder dan meghitung luas alas yang diperlukan.
- State apa yang hanya dimiliki Cylinder?
  - ⇨ State yang hanya dimiliki cylinder yaitu height.

Code :

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder() {
        super();
        height = 1.0;
    }

    public Cylinder(double height) {
        super();
        this.height = height;
    }

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        return getArea() * height;
    }
}
```

Class Cylinder dibuat menggunakan extends Circle dan memiliki tiga seperti pada class circle. Kemudian dilakukan pengujian dengan membuat tiga objjek melalui constructor yang berbeda.

Output & Hasil Pengujian :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCylinder.java
c1
radius     = 1.0
height     = 1.0
base area  = 3.141592653589793
volume     = 3.141592653589793

c2
radius     = 1.0
height     = 10.0
base area  = 3.141592653589793
volume     = 31.41592653589793

c3
radius     = 2.0
height     = 10.0
base area  = 12.566370614359172
volume     = 125.66370614359172

PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Berdasarkan hasil pengujian, ketiga objek berhasil menampilkan output yang diminta termasuk method getRadius() dan getArea() yang dapat dipanggil meskipun tidak ditulis ulang pada Cylinder, tidak seperti getHeight() dan getVolume().

Checkpoint 3 : *Mengapa c3.getRadius() dan c3.getArea() dapat dipanggil padahal method tersebut tidak ditulis ulang pada Cylinder?*

⇨ Karena class Cylinder menggunakan extends Circle, sehingga Cylinder mewarisi method dari superclass Circle yang dapat diakses oleh subclass.

d. Constructor Chain dan super()

Output sementara ditambahkan pada constructor Circle(double radius) dan Cylinder(double radius, double height). Program kemudian dijalankan menggunakan new Cylinder(2.0, 10.0) untuk mengamati urutan pemanggilan constructor.

Output & Hasil Pengujian :

```
Problems 29   Output   Debug Console   Terminal   Ports   SonarQube 29

PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCylinder
Circle(double) dipanggil
Cylinder(double,double) dipanggil
c3
radius     = 2.0
height     = 10.0
base area  = 12.566370614359172
volume     = 125.66370614359172

PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCylinder
Cylinder(double,double) dipanggil
c3
radius     = 1.0
height     = 10.0
base area  = 3.141592653589793
volume     = 31.41592653589793

PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Pada percobaan pertama, constructor Cylinder menggunakan super(radius) untuk memanggil constructor Circle yang menerima radius. Ketika objek Cylinder dibuat, constructor Circle(double radius) dijalankan terlebih dahulu, kemudian dilanjutkan dengan constructor Cylinder(double radius, Height).
Pada percobaan kedua, pemanggil super(radius) dihapus dari constructor Cylinder. Java secara otomatis memanggil constructor superclass tanpa parameter melalui super(). Akibatnya, constructor Circle() yang dijalankan, sehingga nilai radius menjadi 1.0, bukan 2.0

e. Encapsulation pada Subclass

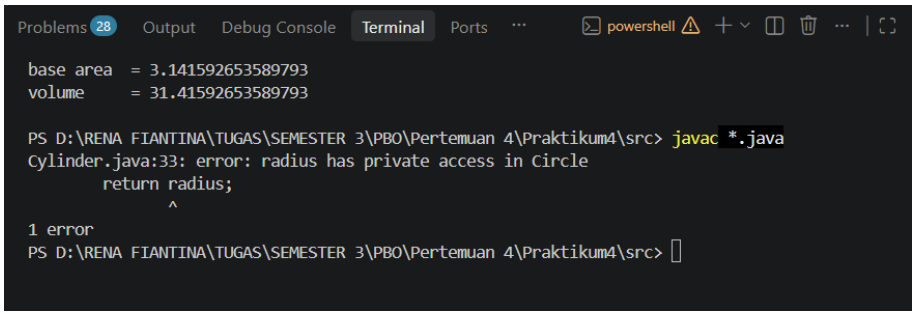
Menambahkan method untuk akses atribut radius yang terdapat pada superclass Circle dari subclass Cylinder.
Code :

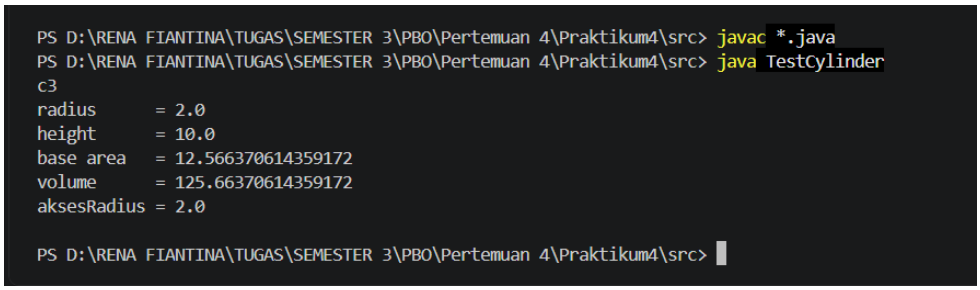
```
// mengakses dengan langsung menggunakan return

public double cobaAksesRadius() {
    return radius;
}


// Perbaikannya
public double cobaAksesRadius() {
    return radius;
}
```

Output & Hasil pengujian





Berdasarkan hasil pengujian Ketika method cobaAksesRadius() dibuat dengan mengembalikan nilai radius secara langsung, program menghasilkan error, karena atribut radius bersifat private. Sehingga atribut private hanya dapat dipanggil secara langsung didalam class nya meskipun Cylinder merupakan subclass.

Perbaikan yang dilakukan yaitu dengan menggunakan method getRadius() yang bersifat public, sehingga cylinder tetap bisa mengambil nilai radius tanpa mengakses atribut private secara langsung.

Checkpoint 5: *extends membuat subclass mewarisi behavior yang dapat diakses, tetapi tidak membuat field private superclass menjadi dapat diakses secara langsung oleh subclass.*

f. Overriding getArea() dan Perbaikan getVolume()

Method getArea() pada class Cylinder dioverride untuk menghitung luas permukaan silinder, yaitu luas selimut ditambah luas dua lingkaran alas dan tutup. Dengan overriding tersebut, pemanggilan getArea() pada objek Cylinder menggunakan implementasi milik Cylinder, bukan implementasi milik Circle

Code :

```
  @Override
  public double getArea() {
     return 2 * Math.PI * getRadius() * height
          + 2 * Math.PI * getRadius() * getRadius();
  }

// getVolume
  public double getVolume() {
      return super.getArea() * height;
  }
```

Output & Hasil Pengujian :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCylinder
c3
radius      = 2.0
height      = 10.0
base area   = 150.79644737231007
volume      = 1507.9644737231006
aksesRadius = 2.0
```

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestCylinder
c3
radius      = 2.0
height      = 10.0
base area   = 150.79644737231007
volume      = 125.66370614359172
aksesRadius = 2.0

PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Method getArea() pada Cylinder di override agar menghitung luas permukaan silinder, yaitu luas selimut ditambah luas dua lingkaran. Namun, method getVolume() masih menggunakan getArea() * height. Akibatnya, perhitungan volume menggunakan luas permukaan silinder, bukan luas alas, sehingga menghasilkan volume yang salah, yaitu 1507.9645 untuk radius 2.0 dan tinggi 10.0

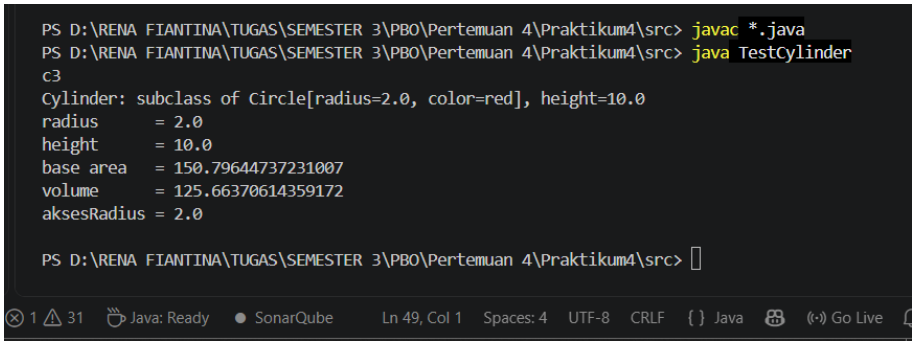
g. Overriding toString() dan @Override

Melakukan overriding terhadap method toString() pada Cylinder dan mencoba pengujian fungsi pada override Ketika nama method salah ditulis.

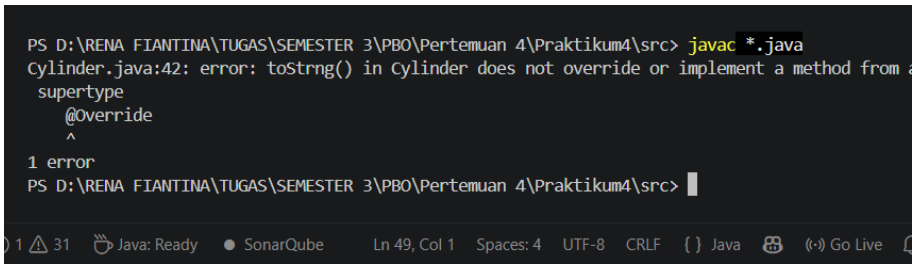
Code :

```
@Override
public String toString() {
    return "Cylinder: subclass of " + super.toString()
          + ", height=" + height;
}
```

Output & Hasil Percobaan :



Method toString() pada Cylinder di override untuk menampilkan superclass melalui super.toString(), kemudian menambahkan atribut height milik Cylinder. Sehingga informasi radius dan warna dari Circle dapat ditampilkan bersama informasi tinggi Cylinder tanpa harus menuliskan ulang code yang sudah ada pada superclass.



Dilakukan pengujian ketika nama method diubah menjadi toStrng() tetapi @Override tidak dihapus. Dengan begitu, compiler menghasilkan error karena method tersebut tidak melakukan overriding terhadap method superclass yang sesuai.
Lalu ketika anotassi @Override dihapus, method tersebut tidak menghasil error ketika di compile dan dianggap sebagai method baru.

Checkpoint 7 : *Tuliskan dengan bahasa Anda: @Override tidak membuat inheritance terjadi. Lalu apa kegunaannya?*

⇨ Karena hubungan inheritance sudah dibuat melalui extends. Kegunaan @Override Adalah membantu compiler memeriksa apakah method yang ditulis pada subclass benar-benar melakukan overriding terhadap yang diwarisi dari superclass.

B. Praktikum II – Shape, Circle, Rectangle, dan Square

a. Pembuatan Superclass Shape

Membuat class umum Bernama Shape yang akan menyimpan atribut dan behavior berdasarkan kesamaan atau kemiripan dari beberapa bentuk, sehingga nantinya bisa digunakan bersamaan oleh subclass lainnya.

Code :

```
public class Shape {
    private String color;
    private boolean filled;

    public Shape() {
        color = "green";
        filled = true;
    }

    public Shape(String color, boolean filled) {
        this.color = color;
        this.filled = filled;
    }

    public String getColor() {
        return color;
    }

    public void setColor(String color) {
        this.color = color;
    }

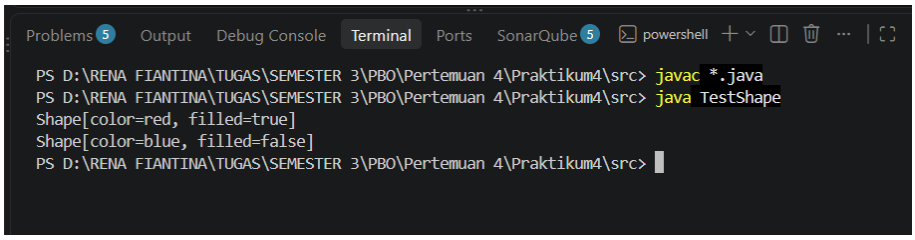
    public boolean isFilled() {
        return filled;
    }

    public void setFilled(boolean filled) {
        this.filled = filled;
    }

    @Override
    public String toString() {
        return "Shape[color=" + color + ", filled=" + filled + "]";
    }
}
```

Class ini memiliki dua constructor, yaitu constructor tanpa parameter dengan nilai default green dan true, dan yang kedua yaitu constructor berparameter untuk menentukan nilai atribut ketika objek dibuat.

Output hasil test code:



Checkpoint 8 : *State apa yang cukup umum sehingga masuk akal dimiliki Circle maupun Rectangle ?*

⇨ State color dan filled, karena kedua atribut tersebut menunjukkan warna dan status suatu bentuk.

b. Inheritance Circle

Menerapkan konsep inheritance dengan menjadikan class shape sebagai superclass dari class Circle, kemudian menambahkan atribut dan method khusus untk lingkaran.

Code :

```
public class Circle extends Shape {
    private double radius;

    public Circle() {
        super();
        radius = 1.0;
    }

    public Circle(double radius) {
        super();
        this.radius = radius;
    }

    public Circle(double radius, String color, boolean filled) {
        super(color, filled);
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }

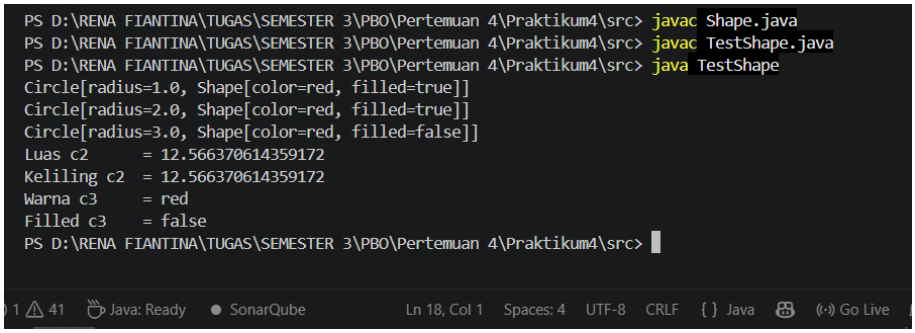
    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public String toString() {
        return "Circle[radius=" + radius + ", " + super.toString() + "]";
    }
}
```

Class Circle menggunakan extends Shape, sehingga mewarisi atribut dan method dari shape. Lalu ada penambahan atribut radius dan method khusus pada class circle. Method toString() di-override agar informasi jari-jari dan atribut dari Shape dapat ditampilkan secara bersamaan.

Output & Hasil Pengujian :



Pengujian dilakukan dengan membuat objek Circle menggunakan tiga constructor berbeda, yaitu constructor tanpa parameter, constructor dengan parameter radius, serta constructor dengan parameter radius, color, dan filled.

Checkpoint : *Perilaku apa yang khusus dimiliki Circle dan tidak perlu dimiliki oleh Shape ?*

- ⇨ getArea() dan getPerimeter() adalah perilaku yang khusus dimiliki Circle untuk mrnghitung luas dan keliling lingkaran. Sedangkan Shape hanya menyimpan color dan filled.

c. Inheritance Rectangle

Membuat subclass Rectangle dari class shape serta menambahkan atribut dan menambahkan atribut dan method khusus untuk Rectangle.

Code :

```
public class Rectangle extends Shape {
    private double width;
    private double length;

    public Rectangle() {
        super();
        width = 1.0;
        length = 1.0;
    }

    public Rectangle(double width, double length) {
        super();
        this.width = width;
        this.length = length;
    }

    public Rectangle(double width, double length,
```

```
                String color, boolean filled) {
        super(color, filled);
        this.width = width;
        this.length = length;
    }

    public double getWidth() {
        return width;
    }

    public void setWidth(double width) {
        this.width = width;
    }

    public double getLength() {
        return length;
    }

    public void setLength(double length) {
        this.length = length;
    }

    public double getArea() {
        return width * length;
    }

    public double getPerimeter() {
        return 2 * (width + length);
    }

    @Override
    public String toString() {
        return "Rectangle[width=" + width
                + ", length=" + length
                + ", " + super.toString() + "]";
    }
}
```

Output dan Hasil Pengujian

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac Shape.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac Rectangle.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac TestShape.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestShape
Rectangle[width=1.0, length=1.0, Shape[color=red, filled=true]]
Rectangle[width=3.0, length=4.0, Shape[color=red, filled=true]]
Rectangle[width=5.0, length=2.0, Shape[color=blue, filled=false]]
Luas r2      = 12.0
Keliling r2  = 14.0
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Checkpoint 10 : *Apa yang dimiliki bersama oleh Circle dan Rectangle, dan apa yang tetap berbeda?*

⇨ Class Circle dan Rectangle memiliki atribut yang sama yaitu color dan filled, serta mewarisi method untuk mengakses dan mengubah atribut tersebut dari superclass Shape. Sementara itu, Class Circle memiliki atribut radius serta method untuk

menghitung luas dan keliling lingkaran. Class Rectangle memiliki atribut width dan length serta method untuk menghitung luas dan keliling persegi panjang.

d. Generalization & Specialization Square

Membuat class Square sebagai subclass dari Rectangle. Class ini menggunakan atribut dan method yang diwarisi dari Rectangle, dengan menambahkan method untuk mengakses dan mengubah nilai sisi persegi. Karena panjang dan lebar persegi harus sama, constructor Square mengatur kedua nilai tersebut menggunakan ukuran sisi yang diberikan.

Code :

```
public class Square extends Rectangle {
    public Square() {
        super(1.0, 1.0);
    }

    public Square(double side) {
        super(side, side);
    }

    public Square(double side, String color, boolean filled) {
        super(side, side, color, filled);
    }

    public double getSide() {
        return getWidth();
    }

    public void setSide(double side) {
        super.setWidth(side);
        super.setLength(side);
    }

    @Override
    public String toString() {
        return "Square[side=" + getSide()
                + ", " + super.toString() + "]";
    }
}
```

Output & Hasil Pengujian :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac Shape.java Rectangle.ja
a Square.java TestShape.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestShape
Square[side=1.0, Rectangle[width=1.0, length=1.0, Shape[color=red, filled=true]]]
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=red, filled=true]]]
Square[side=3.0, Rectangle[width=3.0, length=3.0, Shape[color=blue, filled=false]]]
Luas s2 = 25.0
Keliling s2 = 20.0
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Pengujian dilakukan dengan membuat tiga objek Square menggunakan constructor yang berbeda. Dari output yang muncul, setiap objek memiliki nilai width dan length

yang sama sesuai dengan nilai sisi yang diberikan. Objek juga menampilkan atribut color dan filled yang diwarisi dari Shape.
Pada objek dengan sisi 5.0, luas yang dihasilkan adalah 25.0 dan kelilingnya 20.0. Hasil ini menunjukkan bahwa class Square dapat menggunakan method perhitungan dari Rectangle tanpa perlu membuat ulang method tersebut

e. Pengujian & Pelanggaran aturan Square

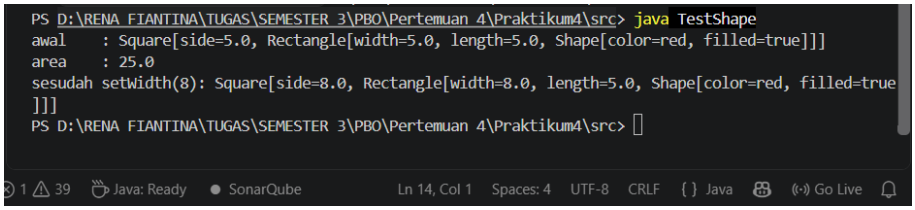
Melakukan pengujian terhadap class Square dengan mengubah nilai width menggunakan method setWidth(). Pengujian ini dilakukan untuk mengetahui apakah perubahan lebar tetap mempertahankan aturan persegi, yaitu panjang dan lebar harus memiliki nilai yang sama.

Code Pengujian :

```
Square s = new Square(5.0);
System.out.println("awal    : " + s);
Modul Praktikum 4 | Inheritance, Generalization & Overriding | Halaman 12
System.out.println("area    : " + s.getArea());

s.setWidth(8.0);
System.out.println("sesudah setWidth(8): " + s);
```

Output & Hasil Pengujian :



Dari output yang muncul, objek Square awalnya memiliki width dan length sebesar 5.0, dengan luas sebesar 25.0. Setelah method setWidth(8.0) dijalankan, nilai width berubah menjadi 8.0, sedangkan length tetap 5.0. dengan begitu panjang dan lebar tidak lagi sama dan objek tersebut tidak memenuhi aturan persegi.
Hal ini dikarenakan method setWidth() yang diwarisi dari Rectangle hanya mengubah nilai width tanpa mengubah length. Oleh karena itu, perlu adanya overriding pada method setter di class Square agar perubahan nilai panjang atau lebar tetap mempertahankan aturan persegi.

f. Override setter pada Square

Melakukan overriding terhadap method setWidth() dan setLength() pada class Square agar perubahan nilai panjang atau lebar tetap mempertahankan aturan persegi, yaitu width dan length harus memiliki nilai yang sama.

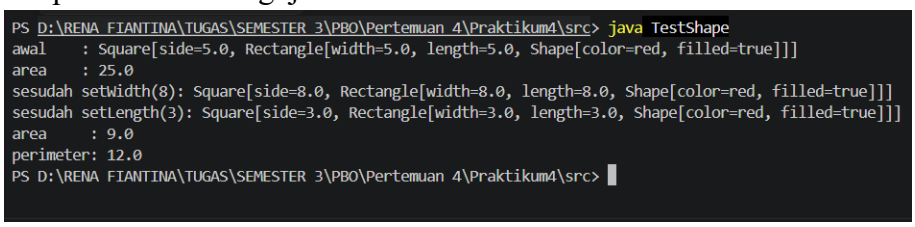
Code :

```
@Override
public void setWidth(double side) {
    super.setWidth(side);
```

```
    super.setLength(side);
}

@Override
public void setLength(double side) {
    super.setWidth(side);
    super.setLength(side);
}
```

Output & Hasil Pengujian :



Berdasarkan hasil pengujian, setelah method setWidth(8.0) dijalankan, nilai width dan length sama-sama berubah menjadi 8.0. Kemudian, ketika method setLength(3.0) dijalankan, kedua nilai berubah menjadi 3.0. Dengan demikian, aturan persegi tetap terpenuhi meskipun nilai panjang atau lebar diubah.
Method getArea() dan getPerimeter() tidak perlu di-override karena perhitungan luas dan keliling dari Rectangle tetap dapat digunakan oleh Square selama nilai width dan length sama.

Checkpoint 12 : *Jelaskan mengapa setWidth()/setLength() perlu dioverride, tetapi getArea()/getPerimeter() dapat tetap memakai implementasi Rectangle*

- ⇨ setWidth() dan setLength() perlu di-override karena method tersebut diwarisi dari Rectangle dan hanya mengubah salah satu nilai, sehingga dapat menyebabkan width dan length berbeda. Dengan overriding, kedua method diubah agar selalu menyesuaikan width dan length dengan nilai sisi yang sama. Sedangkan getArea() dan getPerimeter() dapat tetap menggunakan implementasi Rectangle karena rumus luas dan keliling persegi panjang juga berlaku untuk persegi ketika width dan length bernilai sama.

g. Pengujian Hierarki secara Keseluruhan

Melakukan pengujian terhadap seluruh hierarki class, yaitu Shape, Circle, Rectangle, dan Square. Pengujian dilakukan dengan membuat objek dari masing-masing class untuk melihat hasil inheritance serta penggunaan method yang dimiliki setiap class.

Code :

```
Shape a = new Shape("black", false);
Circle b = new Circle(2.0, "blue", true);
Rectangle c = new Rectangle(3.0, 4.0, "yellow", true);
Square d = new Square(5.0, "green", true);

System.out.println(a);
System.out.println(b + " area=" + b.getArea());
```

```
System.out.println(c + " area=" + c.getArea());
System.out.println(d + " area=" + d.getArea());
```

Output & Hasil Pengujian :

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> javac Shape.java Circle.java Rectangle.java Square.java TestShape.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src> java TestShape
Shape[color=black, filled=false]
Circle[radius=2.0, Shape[color=blue, filled=true]] area=12.566370614359172
Rectangle[width=3.0, length=4.0, Shape[color=yellow, filled=true]] area=12.0
Square[side=5.0, Rectangle[width=5.0, length=5.0, Shape[color=green, filled=true]]] area=25.0
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Praktikum4\src>
```

Berdasarkan hasil pengujian, seluruh objek berhasil menampilkan informasi yang sesuai dengan masing-masing class objek.
Sehingga dari percobaan ini setiap subclass dapat menggunakan atribut dan method yang diwarisi dari superclass serta memiliki atribut dan behavior khusus sesuai karakteristiknya.

C. Kasus Inventaris Barang Kampus

a. Identifikasi Kandidat Generalization

Pada program awal, kelas Barang digunakan untuk merepresentasikan berbagai barang inventaris kampus. Kelas ini dapat dikembangkan dan dikelompokkan menjadi :

- BarangElektronik : laptop dan proyektor
- BarangNonElektronik : meja dan kursi.berbeda

State / behavior yang sama :

- kodeBarang untuk menyimpan kode barang
- nama untuk menyimpan nama barang
- kondisi untuk menyimpan kondisi barang
- getKodeBarang(), getNama(), dan getKondisi() untuk mengakses data barang
- tandaiRusak(), tandaiHilang(), dan perbaiki() untuk mengubah kondisi barang

Perbedaan masing-masing class

b. Hierarchy Class

Barang ditetapkan sebagai superclass, sedangkan BarangElektronik dan BarangNonElektronik menjadi subclass. Kedua subclass mewarisi state dan behavior umum dari superclass serta mengimplementasikan overriding pada method periksaKondisi().

c. Implementasi Perubahan

1. Pada file BarangElektronik.java dan BarangNonElektronik.java, kedua subclass menggunakan extends dan memanggil constructor superclass melalui super(...).

```
public class BarangElektronik extends Barang {
    public BarangElektronik(String kodeBarang, String nama, String kondisi) {
        super(kodeBarang, nama, kondisi);
    }
}
```

```
public class BarangNonElektronik extends Barang {
    public BarangNonElektronik(String kodeBarang, String nama, String kondisi) {
        super(kodeBarang, nama, kondisi);
    }
}
```

2. Method periksaKondisi() di-override pada kedua subclass menggunakan @Override. Pemanggilan super.periksaKondisi() digunakan untuk menjalankan perilaku umum superclass.

```
@Override
public void periksaKondisi() {
    System.out.print("Pemeriksaan barang elektronik: ");
    super.periksaKondisi();
}
```

Pada BarangNonElektronik, implementasinya menggunakan keterangan yang berbeda, yaitu "Pemeriksaan barang non-elektronik: " sebelum memanggil super.periksaKondisi().

3. Atribut kodeBarang, nama, dan kondisi pada superclass tetap menggunakan modifier private. Subclass mengakses kondisi barang melalui method getKondisi(), sehingga enkapsulasi tetap terjaga.

4. Pada Main.java, objek dibuat dari kedua subclass menggunakan referensi bertipe Barang.

```
Barang barang1 = new BarangElektronik(
    "BRG001", "Laptop", "BAGUS"
);

Barang barang3 = new BarangNonElektronik(
    "BRG003", "Meja", "BAGUS"
);

barang1.periksaKondisi();
barang3.periksaKondisi();
```

```
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Inventaris Barang> javac *.java
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Inventaris Barang>  java Main
===== DATA INVENTARIS =====
Kode Pencatatan : PC001
Kode Barang : BRG001
Nama Barang : Laptop
Kode Lokasi : R001

Kode Pencatatan : PC002
Kode Barang : BRG002
Nama Barang : Proyektor
Kode Lokasi : R002

Kode Pencatatan : PC003
Kode Barang : BRG003
Nama Barang : Meja
Kode Lokasi : R002

Kode Pencatatan : PC004
Kode Barang : BRG004
Nama Barang : Kursi
Kode Lokasi : R002

===== PENGUJIAN OVERRIDING =====
Pemeriksaan barang elektronik: Kondisi barang: BAGUS
Pemeriksaan barang non-elektronik: Kondisi barang: BAGUS

===== CHECK BEHAVIOR LAMA =====
Status barang 1 sebelumnya:
BAGUS
Status barang 1 sesudah tandaiRusak():
RUSAK
PS D:\RENA FIANTINA\TUGAS\SEMESTER 3\PBO\Pertemuan 4\Inventaris Barang>
```

D. Refleksi

1. *Apa yang membuat Circle layak menjadi class yang lebih umum daripada Cylinder?*
   Circle merepresentasikan lingkaran yang memiliki jari-jari dan warna, sedangkan Cylinder memiliki karakteristik tersebut namun ditambah tinggi

2. *Berdasarkan Step 5, apa arti praktis extends bagi object Cylinder?*
   extends membuat Cylinder mewarisi atribut dan method dari Circle. Dengan begitu, Cylinder dapat menggunakan fitur seperti getRadius() dan getArea() tanpa harus menuliskannya kembali, sekaligus memiliki atribut dan method khusus seperti tinggi dan volume

3. Mengapa constructor Circle dijalankan ketika Anda membuat object Cylinder?
   Karena Cylinder subclass Circle, bagian superclass harus diinisialisasi terlebih dahulu. Constructor Circle mengatur atribut yang diwarisi, seperti radius dan warna, sebelum constructor Cylinder mengatur atribut tingginya

4. Mengapa getVolume() menjadi salah setelah getArea() dioverride?
   Karena getVolume() pada Cylinder menggunakan getArea() untuk menghitung volume. Setelah getArea() di-override untuk menghitung luas permukaan tabung, hasilnya jadi tidak sesuai dengan luas alas yang dibutuhkan untuk menghitung volume

5. Mengapa Square perlu mengubah setter yang diwarisi dari Rectangle?
   Karena Square memiliki empat sisi yang sama panjang, sedangkan Rectangle memungkinkan lebar dan panjang berbeda. Jika setter Rectangle digunakan tanpa penyesuaian, panjang dan lebar Square dapat berubah secara terpisah sehingga tidak lagi memenuhi sifat persegi.

6. Apakah setiap dua class yang mirip harus menggunakan inheritance? Jelaskan dari salah satu kasus Anda.
   Tidak. Dua class yang mirip belum tentu memiliki hubungan pewarisan yang sesuai. Pada kasus Inventaris Barang Kampus, BarangElektronik dan BarangNonElektronik bisa mewarisi Barang karena keduanya merupakan jenis barang inventaris dan memiliki atribut serta perilaku umum yang sama